

CUSTOMER CHURN INTELLIGENCE

and Retention Decision-Support Platform

End-to-End Machine Learning Model Deployment using Hugging Face and GitHub Actions

Field	Detail
Module	SDAIM — Term 3
Instructor	JP Aggarwal
Team members	Krishna Mathur (AS25DXB018) Yash Petkar (AS25DXB021) Atharva Soundankar (AS25DXB020)
GitHub repository	https://github.com/krish2105/SDAIM-Customer-Churn
Hugging Face Space	https://huggingface.co/spaces/krish21may/churn
Live application	https://krish21may-churn.hf.space
Application version	1.1.0
Deployment status	Live — built, running and verified
Submission date	28 July 2026

All metrics in this report were produced by the project's own code. All screenshots were captured from live, running systems.

Table of Contents

Executive Summary.....	4
What distinguishes this submission.....	4
1. Problem Statement and Business Objective.....	5
1.1 The business problem.....	5
1.2 The decision this platform supports.....	5
1.3 Stakeholders.....	5
1.4 Required outputs.....	5
1.5 Governance boundary.....	5
2. Dataset Description and Source.....	6
2.1 Source and provenance.....	6
2.2 Why the dataset was chosen.....	6
2.3 Fictional-data disclosure.....	6
2.4 Integrity verification.....	6
2.5 Features.....	7
3. Exploratory Data Analysis and Key Observations.....	8
3.1 Target balance.....	8
3.2 Data quality.....	8
3.3 Contract term is the strongest categorical signal.....	9
3.4 Payment method and internet service.....	10
3.5 Tenure.....	11
3.6 Numeric correlations.....	11
3.7 Key observations carried into modelling.....	12
4. Data Preprocessing.....	13
4.1 Steps applied.....	13
4.2 Data-leakage prevention.....	13
4.3 Handling unseen categories at inference.....	13
4.4 Feature engineering.....	14
5. Models Trained and Performance Comparison.....	15
5.1 Models compared.....	15
5.2 Evaluation metrics and why they were chosen.....	15
5.3 Cross-validated comparison.....	16
5.4 Full performance comparison.....	16
5.5 Why accuracy alone would have been misleading.....	17
5.6 Confusion matrix of the selected model.....	17
5.7 What the errors mean commercially.....	18
6. Justification for the Selected Model.....	19
6.1 The selection rule, fixed in code before any test result existed.....	19
6.2 What the rule actually did.....	19
6.3 An honest note for examination.....	19
6.4 Supporting reasons for the linear model.....	19
6.5 Saved artifact and reproducibility.....	19
7. Model Governance Analysis.....	21
7.1 Fairness audit.....	21
Findings.....	21
The counterfactual that changed the recommendation.....	22
7.2 Probability calibration.....	22
7.3 Decision threshold.....	23
7.4 Per-prediction explainability.....	24

7.5 Drift detection.....	24
7.6 Feature engineering and hyperparameter search.....	25
8. The Working Application.....	27
8.1 Design principles.....	27
8.2 Application home screen.....	27
8.3 Input form and logical dependencies.....	27
8.4 A completed assessment.....	28
8.5 Per-prediction contribution analysis.....	29
8.6 Batch scoring and the retention work queue.....	30
8.7 Local testing performed.....	30
8.8 A note on the evidence screenshots.....	30
9. GitHub Repository Structure.....	32
9.1 Repository.....	32
9.2 Structure.....	32
9.3 The deployment package.....	33
9.4 Version control practice.....	34
10. The GitHub Actions Workflows.....	35
10.1 Continuous integration — ci.yml.....	35
10.2 Deployment — deploy.yml.....	35
10.3 Successful workflow execution.....	36
10.4 The guard demonstrated failing before it was demonstrated passing.....	38
11. Deployment on Hugging Face.....	39
11.1 The Space.....	39
11.2 Space configuration.....	39
11.3 The container.....	40
11.4 Verified deployment evidence.....	40
12. How the Automated Deployment Works.....	42
12.1 The chain of events.....	42
12.2 Authentication.....	42
12.3 The visible-change test.....	42
13. Security and Governance.....	44
13.1 Credential handling.....	44
13.2 Secret scanning, and its own verification.....	44
13.3 Container hardening.....	44
13.4 Application-level controls.....	44
13.5 Governance position.....	44
13.6 Known security gaps.....	45
14. Testing and Quality Assurance.....	46
14.1 Test suite.....	46
14.2 Tests that guard against silent corruption.....	46
14.3 Quality gates.....	46
14.4 Defects found and fixed by these gates.....	46
15. Limitations and Future Work.....	48
15.1 Limitations.....	48
15.2 Future work.....	48
16. Conclusion.....	49
17. References.....	49
18. Team Contributions.....	49

Executive Summary

This project delivers a working customer-retention decision-support platform: a validated dataset, a leakage-safe machine-learning pipeline, an interactive Streamlit application, Docker packaging, and fully automated deployment to a public Hugging Face Space through GitHub Actions. **Every component has been built, tested and deployed. The application is live and can be used now.**

The selected model — Logistic Regression inside a complete scikit-learn pipeline — achieves **0.8414 ROC-AUC** and **0.7834 recall** on a held-out test set of 1,409 customers that no model saw during fitting or selection. It identifies 293 of the 374 actual churners in that set and misses 81.

Recall is weighted deliberately. A missed churner receives no retention review at all, and the opportunity to intervene is lost silently. An unnecessary review costs a specialist's time and is recoverable. The two errors are not equal, and the model is tuned for the one that matters.

What distinguishes this submission

- **Provenance is proven, not asserted.** The dataset's Git blob SHA is verified against the official IBM publication on every run, in local development and in continuous integration.
- **The model selection rule was fixed in code before the test set was examined.** This is what makes the held-out evaluation genuinely unbiased.
- **A fairness audit was performed, with a measured counterfactual.** Removing the protected attributes was found to cost only +0.0008 ROC-AUC, which changed the recommendation.
- **Calibration was predicted, then measured.** The model was expected to be over-confident because of balanced class weights; it is, by 0.15.
- **Feature engineering and hyperparameter search were tried and rejected on the evidence.** The best gain was +0.0022 against a pre-declared bar of 0.005.
- **115 automated tests** and ten local quality gates, all passing.

Scope of claims. The dataset is IBM's fictional telecommunications sample. This platform demonstrates technical feasibility and may support prioritisation of accounts for human review. No revenue, churn-reduction or return-on-investment effect has been measured, and none is claimed anywhere in this report.

1. Problem Statement and Business Objective

1.1 The business problem

A telecommunications organisation loses customers continuously, but its retention specialists have finite time. Without a consistent basis for deciding which accounts deserve attention, prioritisation falls to individual judgement: inconsistent between specialists, undocumented, and impossible to audit or improve.

The organisation does not need a system that decides what to do about a customer. It needs a repeatable, documented estimate of which accounts are at elevated risk, so that a human being can review the right ones first.

1.2 The decision this platform supports

Decision supported: prioritising customer accounts for a structured human retention review.

Explicitly not supported: changing prices, modifying or terminating contracts, denying or degrading service, or making any automated customer-treatment decision.

1.3 Stakeholders

Stakeholder	Interest in this platform
Chief Customer Officer	Consistency and defensibility of retention prioritisation
Chief Marketing Officer	Targeting efficiency of retention activity
Retention leadership	A bounded, prioritised review workload
Customer-service managers	Practical account-level guidance
Data and analytics	Reproducibility and maintainability
Model risk and governance	Documented limitations and human-review controls

1.4 Required outputs

For a single customer record the platform returns a predicted churn class, a churn probability, a Low / Medium / High risk band, a cautiously worded suggested review action, the model version, and an explicit statement of the model's limitations.

1.5 Governance boundary

Human review is mandatory. Every output of this model requires human review before any customer is contacted. The model has no authority to act, and no downstream system consumes its output as an automated trigger. This constraint appears in the model card, on screen in the application, and in this report because it is a design property, not a disclaimer.

2. Dataset Description and Source

2.1 Source and provenance

Property	Value
Dataset	IBM Telco Customer Churn sample
Publisher	IBM
Official repository	https://github.com/IBM/telco-customer-churn-on-icp4d
Licence	Apache-2.0
Records	7,043 customers
Columns	21
Target variable	Churn (Yes / No)
Identifier excluded from modelling	customerID
Verified Git blob SHA	3de7a612d1609f25f21a455bda77948729369002
SHA-256 (informational)	16320c9c1ec72448db59aa0a26a0b95401046bef5d02fd3aeb906448e3055e91

2.2 Why the dataset was chosen

The problem is a binary classification task with a clear business decision attached, a well-defined target, and enough records and features for meaningful analysis. The dataset is officially published, permissively licensed, and — critically — verifiable: its exact revision can be proven rather than assumed.

2.3 Fictional-data disclosure

Important. The IBM sample represents a fictional telecommunications company. It is realistic in structure and officially published, but it is not actual observed commercial customer data, and it is never described as such anywhere in this project. The brief asks for a real-world dataset; this point was raised with the instructor and is recorded as pending confirmation in PROJECT_INPUTS.md and as decision D-01 in DECISIONS.md.

2.4 Integrity verification

The raw file is treated as immutable evidence. It is never written to, and its identity is proven on every run rather than trusted:

```
$ git hash-object data/raw/Telco-Customer-Churn.csv
3de7a612d1609f25f21a455bda77948729369002

$ make validate
Rows / columns : 7043 / 21
Git blob SHA   : 3de7a612d1609f25f21a455bda77948729369002
Expected SHA   : 3de7a612d1609f25f21a455bda77948729369002
```

```
Target counts    : {'No': 5174, 'Yes': 1869}
Checks executed  : 30
RESULT           : PASSED
```

Thirty automated checks cover column names and order, row count, identifier uniqueness, target and category domains, numeric convertibility, impossible negative values, blank-value counts and the blob SHA. Continuous integration runs the validator with `--strict-sha`, so a missing or altered dataset fails the build rather than producing quietly different results.

2.5 Features

Nineteen predictors are used: sixteen categorical and three numeric. The identifier is excluded entirely.

Group	Features
Customer profile	gender, SeniorCitizen, Partner, Dependents
Services	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies
Contract and billing	Contract, PaperlessBilling, PaymentMethod
Charges and tenure	tenure, MonthlyCharges, TotalCharges

`SeniorCitizen` is encoded 0/1 but is a binary flag, not a magnitude, so it is treated as a category. Scaling it as a number would imply an ordering and a distance that do not exist.

3. Exploratory Data Analysis and Key Observations

Every figure and number in this section is produced by `src/eda.py` from the validated raw file. None was entered by hand. The script is the single source of truth; the accompanying notebook imports the same functions rather than duplicating the logic, so the two cannot disagree.

3.1 Target balance

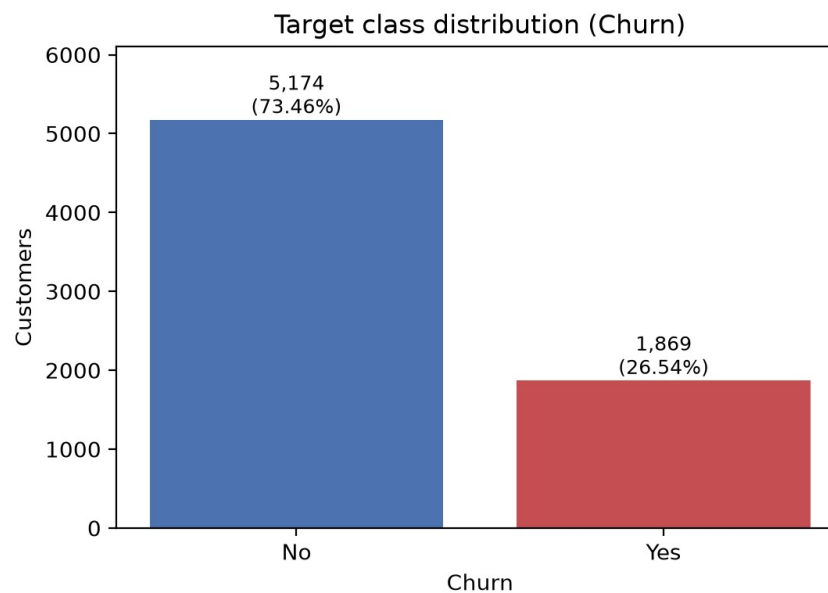


Figure 1. Target class distribution. 1,869 customers churned (26.54%) against 5,174 retained.

The sample contains 26.54% churners. This single fact drives the entire evaluation strategy: a model that predicted "retained" for every customer would score **73.46% accuracy while identifying nobody at risk**. Accuracy alone is therefore not an acceptable selection metric, and recall, F1 and ROC-AUC are used instead.

3.2 Data quality

After stripping whitespace and coercing TotalCharges to numeric, exactly 11 values are missing across the entire table — all in TotalCharges, and all belonging to customers with tenure = 0. These are people who have not yet completed a billing cycle. The blank is structurally meaningful rather than random, which is why the rows are retained rather than dropped.

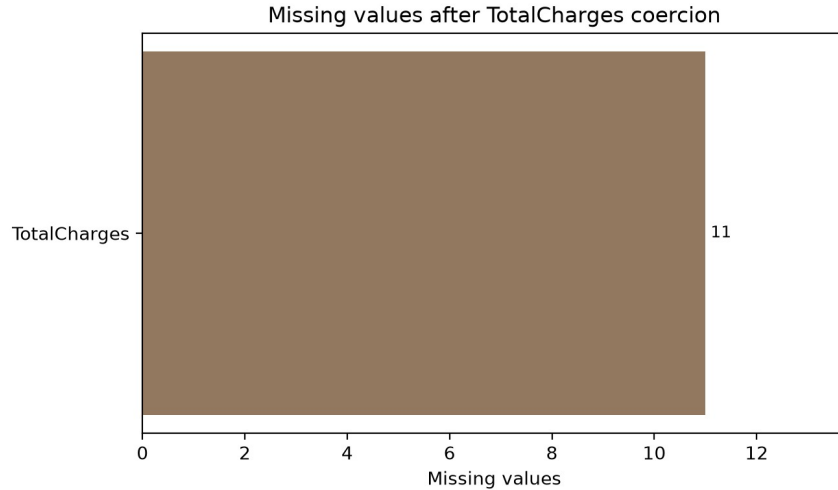


Figure 2. Missing values after coercion. All 11 are in TotalCharges and all have tenure = 0.

There are no duplicate rows and no duplicate customerID values. No numeric field contains a negative value. All sixteen categorical fields match the domains in the IBM data dictionary exactly.

3.3 Contract term is the strongest categorical signal

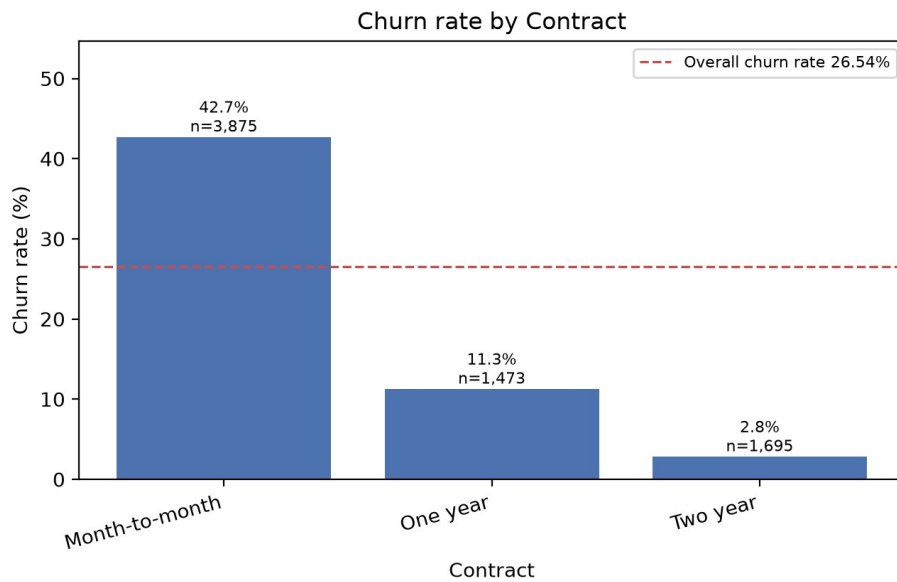


Figure 3. Churn rate by contract term. Month-to-month customers churn at more than fifteen times the two-year rate.

Contract term	Customers	Churn rate
Month-to-month	3,875	42.71%
One year	1,473	11.27%
Two year	1,695	2.83%

3.4 Payment method and internet service

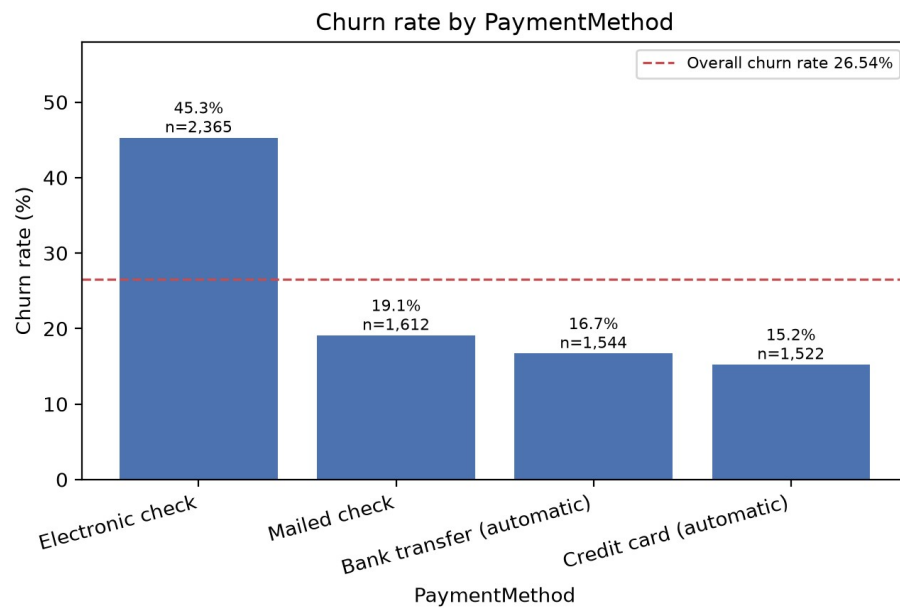


Figure 4. Churn rate by payment method. Electronic check customers churn at nearly three times the automatic-credit-card rate.

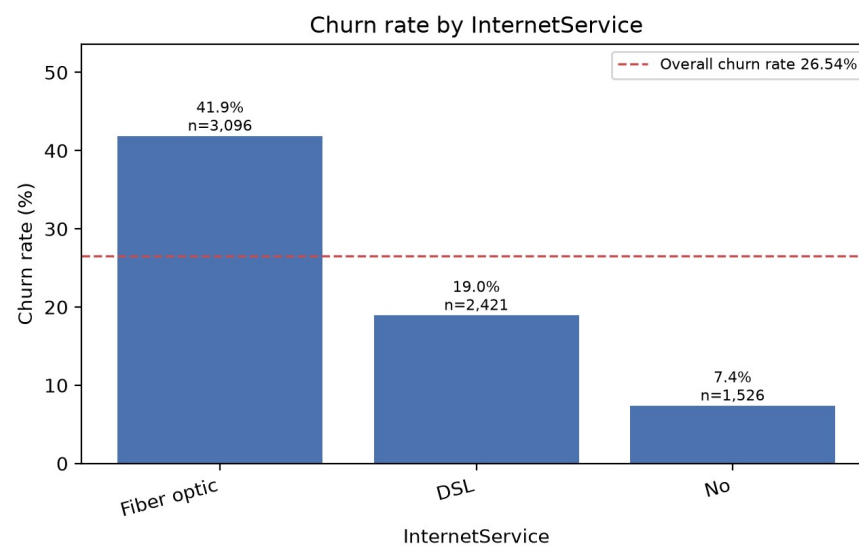


Figure 5. Churn rate by internet service. Fibre optic customers churn at 41.89% against 7.40% for customers with no internet service.

3.5 Tenure

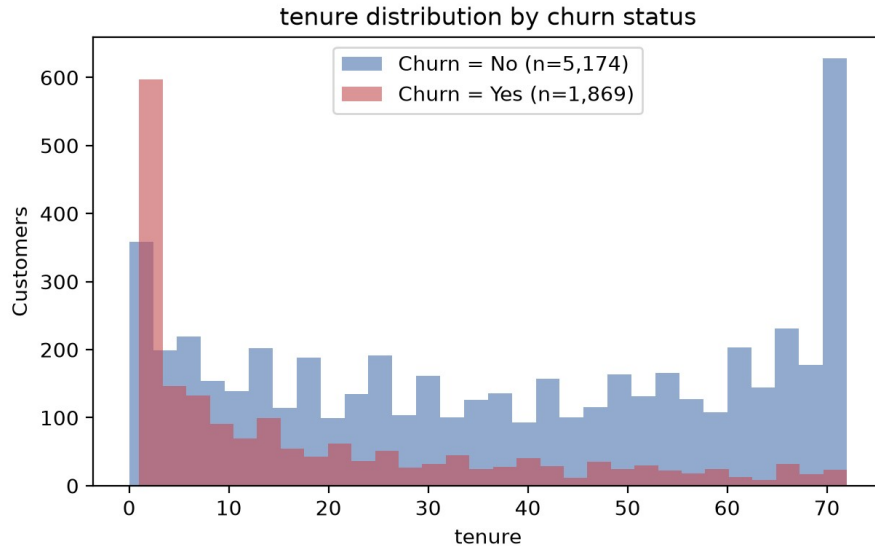


Figure 6. Tenure distribution by churn status. Churn concentrates sharply among recently acquired customers.

Churned customers average 17.98 months of tenure against 37.57 months for retained customers. Median tenure is 10 months for churners and 38 for non-churners. Early-life accounts carry materially higher risk.

3.6 Numeric correlations

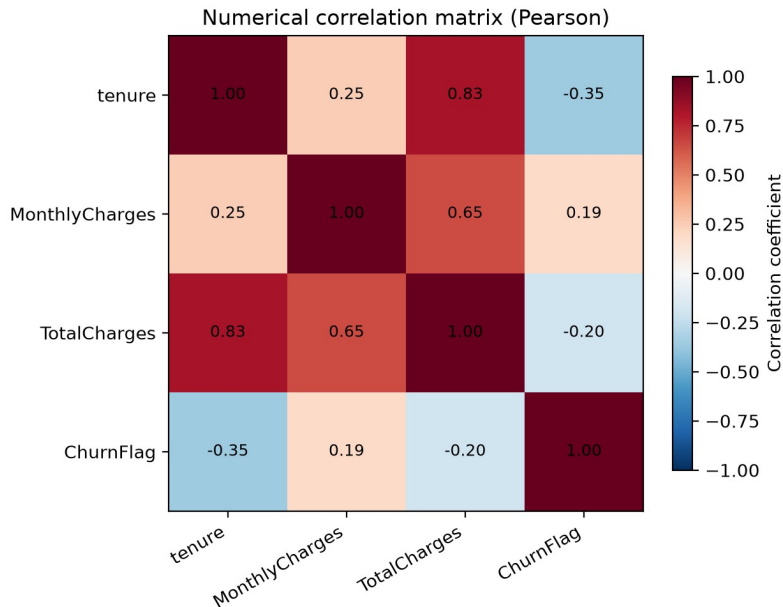


Figure 7. Pearson correlation between the numeric predictors and the binary churn flag.

Tenure correlates with the churn flag at -0.352 , MonthlyCharges at $+0.193$ and TotalCharges at -0.200 . Tenure and TotalCharges correlate strongly with each other at 0.826 , which is expected because total charges

accumulate with time. This is recorded as a known limitation: the tree ensemble tolerates it, and the linear model relies on scaling and regularisation.

3.7 Key observations carried into modelling

1. Class imbalance means recall and ROC-AUC drive model selection, not accuracy.
2. TotalCharges requires coercion and in-pipeline imputation; the raw file stays untouched.
3. SeniorCitizen is a binary category, not a continuous magnitude.
4. customerID is excluded from the feature set entirely.
5. Contract term, internet service, tenure and payment method are the strongest observed signals and are all retained.

Association, not causation. Every relationship described above is an association within a fictional sample. None is evidence of cause. The service add-on comparisons are additionally confounded, because the level "No internet service" appears in every add-on column and therefore partly restates the internet-service split.

4. Data Preprocessing

4.1 Steps applied

6. **Type correction.** TotalCharges is stripped of whitespace and coerced with errors="coerce"; tenure and MonthlyCharges are cast to numeric.
7. **Target mapping.** Churn is mapped Yes to 1 and No to 0, outside the feature matrix.
8. **Identifier exclusion.** customerID is dropped before X is constructed. Two assertions in the code make this impossible to undo by accident.
9. **Type discipline.** SeniorCitizen is cast to string so it routes through the categorical branch.
10. **Single stratified split.** 80/20, random_state=42, performed once — before any transformer exists.
11. **Numeric branch.** Median imputation, plus standard scaling for the linear model only.
12. **Categorical branch.** Most-frequent imputation, then one-hot encoding with handle_unknown="ignore".
13. **Single pipeline.** Preprocessing and estimator are combined into one scikit-learn Pipeline and saved as one artifact.

4.2 Data-leakage prevention

The leakage control in this project is **structural rather than procedural**. It does not depend on anyone remembering to do things in the right order.

- The split happens once, before any imputer, encoder or scaler is instantiated.
- Every transformer lives inside the Pipeline, so cross-validation refits each of them from scratch on every fold. No statistic computed on held-out rows can reach a fitted parameter.
- The held-out test set is used exactly once, after model selection is complete.
- Stratification preserves the class balance: the positive rate is 0.2654 in both the training and test splits.

The concrete mistake this prevents. Imputing the 11 missing TotalCharges values before splitting would compute the median using test-set rows. The model would then have been tuned against information it will not have in production, and every reported metric would be optimistic by an unknown amount.

4.3 Handling unseen categories at inference

OneHotEncoder(handle_unknown="ignore") turns an unseen category into an all-zero block rather than raising. The deployed application therefore degrades gracefully instead of crashing, and still returns a prediction for human review. A test scores a record containing an invented payment method and contract type to confirm this.

4.4 Feature engineering

Seven interpretable features were constructed and evaluated: average realised monthly spend, a charges trend ratio, counts of services and protective add-ons, tenure bands, a new-customer flag and a no-protection flag. Section 7.6 reports the outcome, which was that they did not earn their place.

5. Models Trained and Performance Comparison

5.1 Models compared

Model	Role	Configuration
Logistic Regression	Candidate	max_iter=2000, class_weight="balanced", with median imputation, standard scaling and one-hot encoding
Random Forest	Candidate	400 trees, min_samples_leaf=5, class_weight="balanced", with median imputation and one-hot encoding
Dummy (stratified)	Reference only	Never eligible for selection; exists to make the accuracy argument concrete

The two candidates are deliberately complementary — a regularised linear model and a bagged tree ensemble — both explainable and both available without adding a dependency. Gradient-boosting libraries were considered and rejected: they add a compiled dependency to the deployment image, and the marginal gain would not change any conclusion in this report.

5.2 Evaluation metrics and why they were chosen

Metric	Why it matters here
Recall	The share of actual churners identified. A miss means no review happens at all — the costly, silent error.
Precision	The share of flagged accounts that really churn. Governs wasted specialist time.
F1	Balances the two on the minority class.
ROC-AUC	Threshold-independent ranking quality — the right measure for a prioritisation tool.
Accuracy	Reported for completeness, but insufficient alone on a 26.54% positive rate.

5.3 Cross-validated comparison

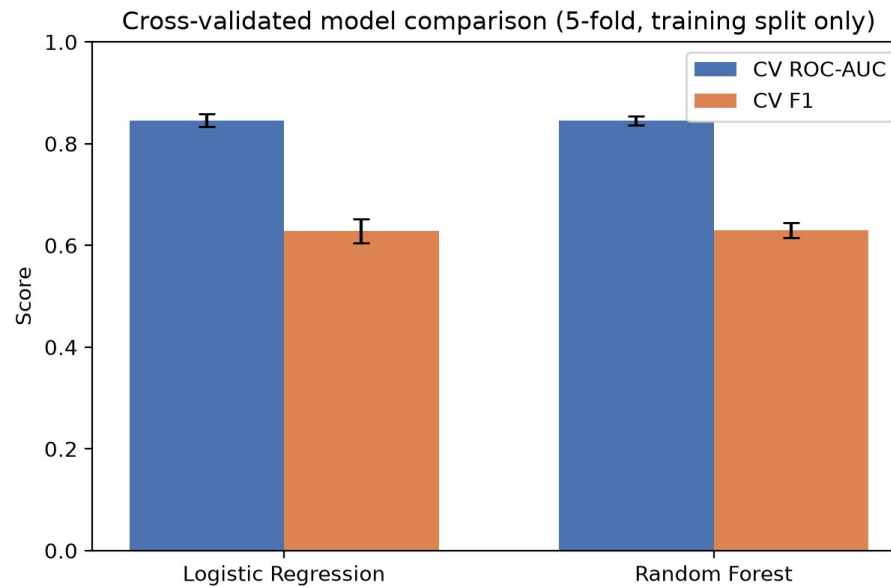


Figure 8. Cross-validated model comparison, five-fold stratified, on the training split only.

5.4 Full performance comparison

Cross-validation columns are computed on the training split only. Test columns come from the held-out 20% that no model saw during fitting or selection.

Model	CV ROC-AUC	CV F1	CV recall	Test ROC-AUC	Test F1	Test recall	Test precision	Test accuracy
Logistic Regression (selected)	0.8460	0.6286	0.8013	0.8414	0.6130	0.7834	0.5034	0.7374
Random Forest	0.8454	0.6296	0.7632	0.8417	0.6349	0.7834	0.5337	0.7608
Dummy (stratified)	0.5065	0.2762	0.2776	0.5163	0.2903	0.2914	0.2891	0.6217

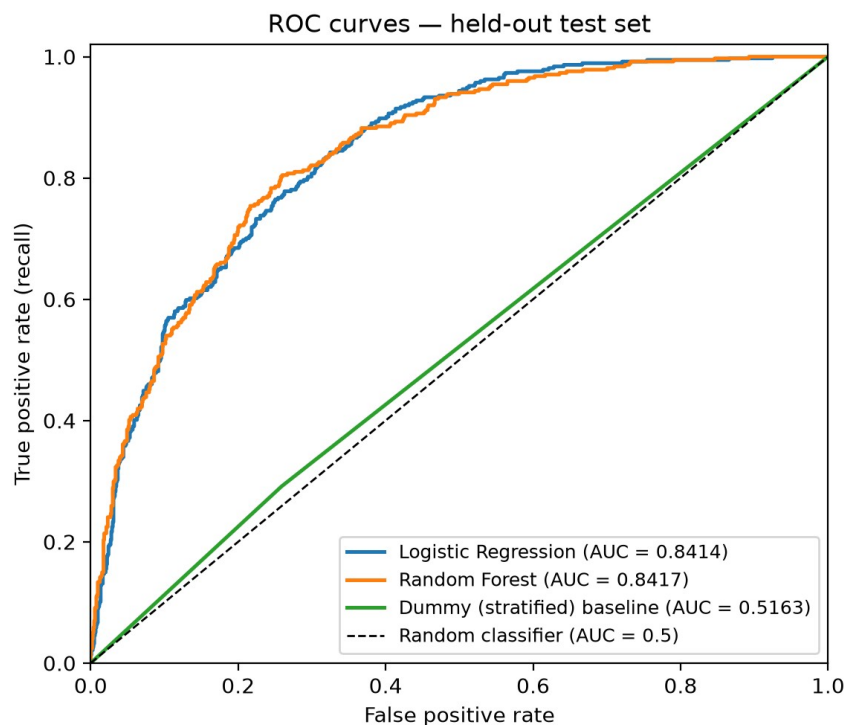


Figure 9. ROC curves on the held-out test set. Both candidates reach approximately 0.841 AUC; the stratified baseline sits on the diagonal.

5.5 Why accuracy alone would have been misleading

The stratified baseline makes this concrete rather than rhetorical. It achieves **62.17% accuracy** while finding almost no churners: recall 0.2914 and ROC-AUC 0.5163, which is no better than chance at ranking. An accuracy-led selection would have rated it as respectable.

5.6 Confusion matrix of the selected model

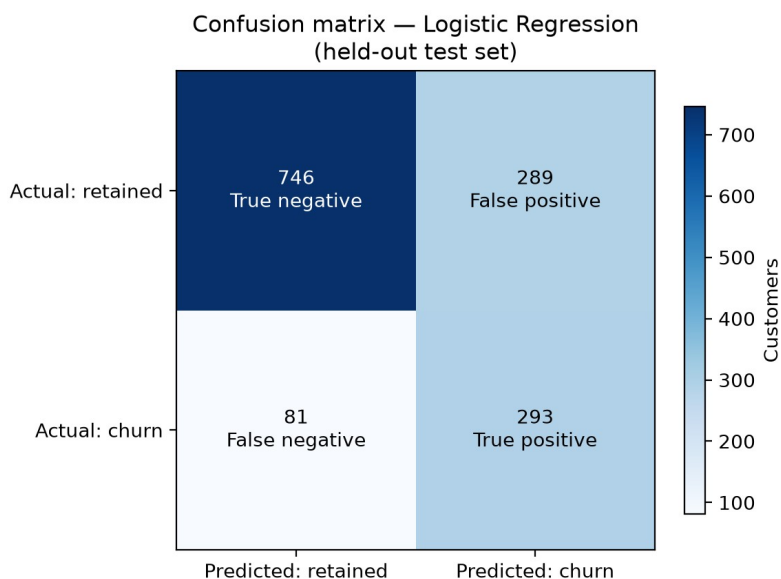


Figure 10. Confusion matrix for the selected model on the held-out test set of 1,409 customers.

	Predicted: retained	Predicted: churn
Actual: retained	746 (true negative)	289 (false positive)
Actual: churn	81 (false negative)	293 (true positive)

5.7 What the errors mean commercially

- **81 false negatives of 374 actual churners (21.7%).** These customers churned but were never flagged. No retention review happens, so the opportunity is lost silently. This is the costly error and the reason recall is weighted.
- **289 false positives of 582 flagged accounts (49.7%).** These customers would have stayed. The cost is specialist time and the risk of an unnecessary contact — recoverable.
- **582 of 1,409 customers flagged (41.3%).** The review workload is bounded and plannable.

6. Justification for the Selected Model

6.1 The selection rule, fixed in code before any test result existed

The rule below was written into `src/train.py` as executable code before the test set was scored. This is what makes the held-out evaluation unbiased: a rule chosen after seeing test numbers is not a rule.

1. Compare candidates by mean ROC-AUC under stratified 5-fold cross-validation on the TRAINING SPLIT ONLY.
2. If the two means differ by less than 0.01, break the tie on mean CV F1.
3. If that gap is also under 0.01, break the tie on mean CV recall, because a missed churner costs more than an unnecessary review.
4. Score the held-out test set ONCE, after selection, for reporting only.

6.2 What the rule actually did

The two candidates were effectively tied. Cross-validated ROC-AUC differed by 0.0005 (0.8460 against 0.8454) and F1 by 0.0010 (0.6286 against 0.6296). Both gaps fell inside the 0.01 tolerance, so the rule fell through to cross-validated recall, where Logistic Regression led 0.8013 to 0.7632.

Logistic Regression was therefore selected.

6.3 An honest note for examination

Stated openly rather than omitted. On the held-out test set the Random Forest is marginally better on accuracy (0.7608 against 0.7374), precision (0.5337 against 0.5034) and F1 (0.6349 against 0.6130), with identical recall (0.7834) and effectively identical ROC-AUC (0.8417 against 0.8414). The model was not switched. Reselecting after seeing test results would convert the held-out set into a selection set and invalidate every unbiasedness claim in this report. The two models are close enough that either is defensible; what is not defensible is choosing after peeking.

6.4 Supporting reasons for the linear model

- Its coefficients yield an exact per-prediction contribution decomposition, used in Section 7.4. No approximation is required.
- It produces a smaller artifact (8.4 KB) and faster inference, which matters for a container that must start quickly.
- It is far easier to defend to a model-risk reviewer than an ensemble of 400 trees.

6.5 Saved artifact and reproducibility

Artifact	Contents
model_pipeline.joblib	The complete pipeline — preprocessing and estimator together, 8.4 KB
model_metadata.json	Model name, version, training timestamp, seed, dataset blob SHA, metrics, confusion matrix, environment versions

Artifact	Contents
feature_schema.json	Predictors, dtypes, observed training categories, numeric bounds, target mapping, UI labels and grouping
reference_rates.json	Descriptive statistics used only for the application's context charts
model_card.md	Intended use, out-of-scope uses, provenance, metrics, limitations, ethics, human-review requirement

Saving the whole pipeline rather than the estimator alone means the deployed transformation cannot differ from the trained one. The artifact is 8.4 KB, so **Git LFS is not required** — this was measured, not assumed. Reproducibility is guaranteed by a fixed seed, pinned dependency versions and the verified dataset SHA; retraining produces byte-identical metrics.

7. Model Governance Analysis

Version 1.1.0 published a list of its own limitations. This section reports the work done to close them, and is the part of the project that goes furthest beyond the brief's minimum.

7.1 Fairness audit

gender and SeniorCitizen are model predictors, so a subgroup audit was performed on the held-out test set. Three standard criteria were computed. They are mathematically incompatible whenever group base rates differ, so a choice was forced and justified.

This project optimises for equal opportunity — equal recall across groups — because the harm that matters is an at-risk customer being missed entirely.

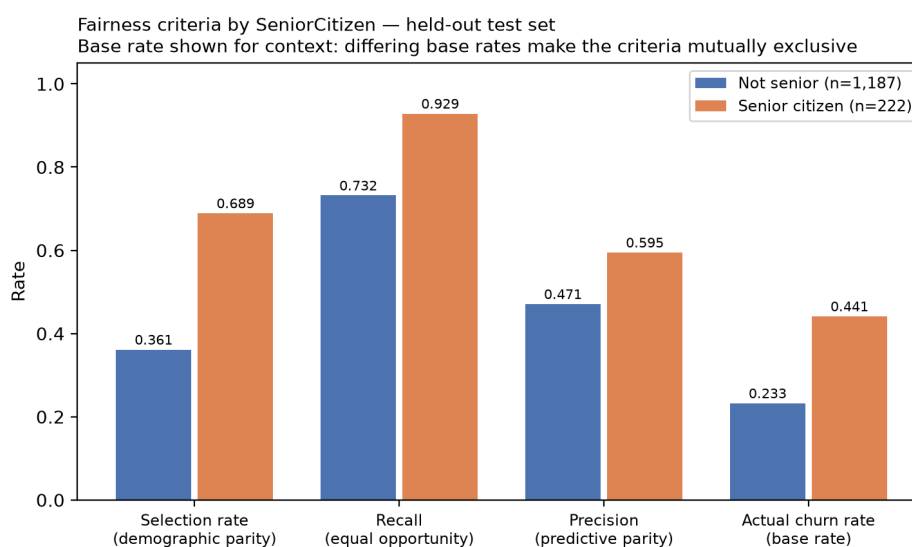


Figure 11. Fairness criteria by SeniorCitizen on the held-out test set, with group base rates shown for context.

Group	n	Actual churn rate	Selection rate	Recall	Precision
Not senior	1,187	0.2325	0.3614	0.7319	0.4709
Senior citizen	222	0.4414	0.6892	0.9286	0.5948
Female	687	0.2809	0.4134	0.7772	0.5282
Male	722	0.2507	0.4127	0.7901	0.4799

Findings

- **gender:** no disparity exceeded the 0.10 materiality convention on any criterion.
- **SeniorCitizen:** material disparity on all four criteria — but the direction matters. Seniors genuinely churn far more (0.4414 against 0.2325), and the model finds at-risk seniors better (recall 0.9286 against 0.7319). On the criterion this project optimises for, the group under-served is non-seniors. The genuine

burden on seniors is the false-positive rate (0.5000 against 0.2492): unnecessary contact, not missed reviews.

The counterfactual that changed the recommendation

Rather than argue from principle, the model was retrained with both protected attributes removed and the cost measured:

Metric	With attributes	Without	Cost of removal
ROC-AUC	0.8414	0.8406	+0.0008
Recall	0.7834	0.7807	+0.0027
F1	0.6130	0.6102	+0.0028

Recommendation: remove both attributes before any operational use. They buy no measurable accuracy — the loss is well inside the ± 0.0124 cross-validation standard deviation — while introducing a disparity that must then be explained and defended. They are retained in version 1.1.0 only to preserve the published evidence trail for this submission. A group-specific decision threshold was considered and rejected outright as direct differential treatment by protected characteristic.

7.2 Probability calibration

The application displays a probability to a decision-maker. A model can rank customers well while its probabilities are systematically wrong, and only calibration justifies presenting a number as though it means what it says.

The outcome was predicted before it was measured. Logistic regression is normally well calibrated, but this model uses `class_weight="balanced"`, which deliberately inflates minority-class probabilities to raise recall. It was therefore expected to be over-confident about churn — and it is.

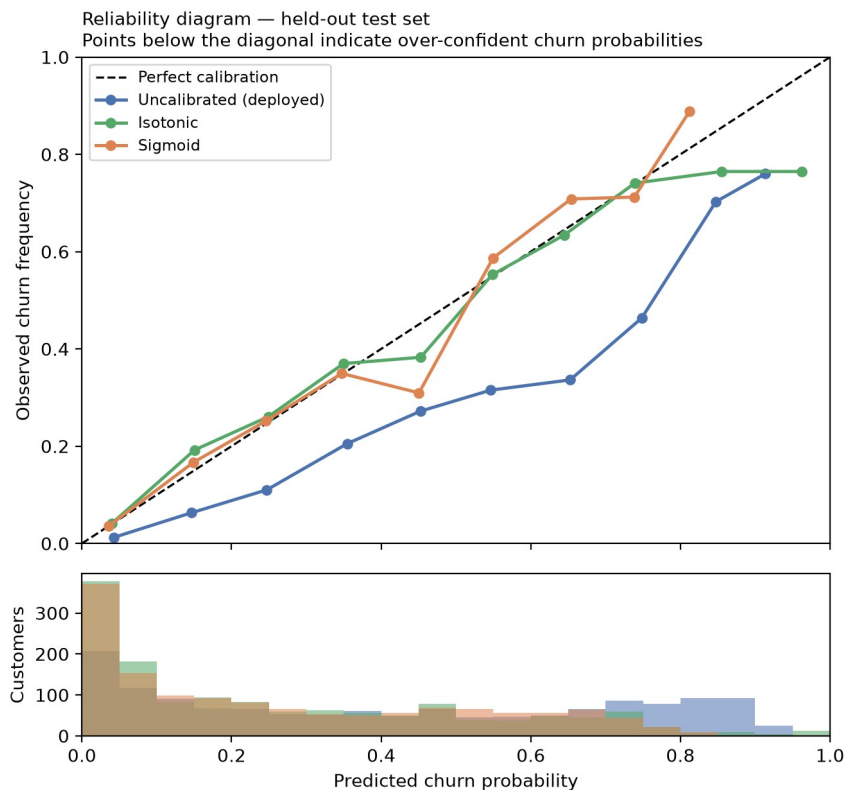


Figure 12. Reliability diagram on the held-out test set, with the distribution of predicted probabilities beneath.

Variant	Brier	ECE	MCE	ROC-AUC	Mean predicted
Deployed (uncalibrated)	0.1688	0.1503	0.3163	0.8414	0.4157
Isotonic	0.1388	0.0194	0.1974	0.8413	0.2671
Sigmoid	0.1383	0.0254	0.1410	0.8416	0.2678

The observed base rate is 0.2654 against a mean prediction of 0.4157 — the model overstates churn probability by roughly 0.15. Isotonic calibration reduces Expected Calibration Error from 0.1503 to 0.0194 while leaving ROC-AUC unchanged at 0.8413, confirming that calibration changes what the number means, not who is ranked highest. The application states on screen that its probabilities are model scores rather than validated frequencies.

7.3 Decision threshold

Version 1.1.0 used a threshold of 0.50 and documented it honestly as a default rather than an optimum. The cost-optimal threshold depends entirely on one business quantity — how many unnecessary reviews one missed churner is worth — which was never supplied. Inventing a currency figure would have been fabricating a business claim, so the whole sensitivity curve is published instead.

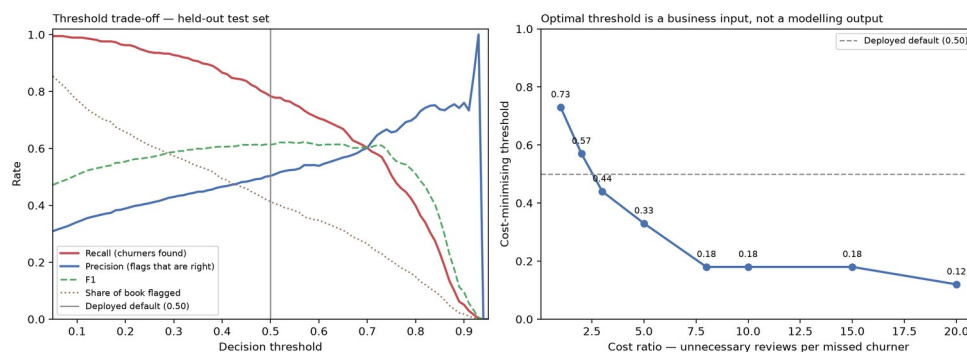


Figure 13. Threshold trade-off, and the cost-minimising threshold as a function of the miss-to-review cost ratio.

Cost ratio	Optimal threshold	Recall	Precision	Customers flagged
1:1	0.73	0.5695	0.6574	324 (23.0%)
3:1	0.44	0.8422	0.4817	654 (46.4%)
5:1	0.33	0.9171	0.4403	779 (55.3%)
10:1	0.18	0.9733	0.3828	951 (67.5%)
20:1	0.12	0.9866	0.3545	1,041 (73.9%)

The deployed 0.50 threshold is cost-optimal at a ratio of roughly **3:1**. Using it was always an implicit assertion about relative costs; that assertion is now explicit. The application exposes the threshold as an adjustable control so the sensitivity is visible to the user rather than buried in a report.

7.4 Per-prediction explainability

For a linear model, each feature's contribution to the log-odds is exactly its coefficient multiplied by its transformed value. This is **not an approximation of the model — it is the model**, and the contributions plus the intercept reconstruct the score exactly. A test asserts that reconstruction to floating-point precision across 25 real customers on every run. SHAP was considered and rejected: it would estimate, at the cost of a heavy dependency, a quantity available here in closed form.

7.5 Drift detection

What this is, and what it is not. The dataset is a single cross-section with no time dimension, so no real drift can be observed and none is claimed. What has been built is the apparatus — a persisted baseline profile and two detectors — together with a demonstration that it works.

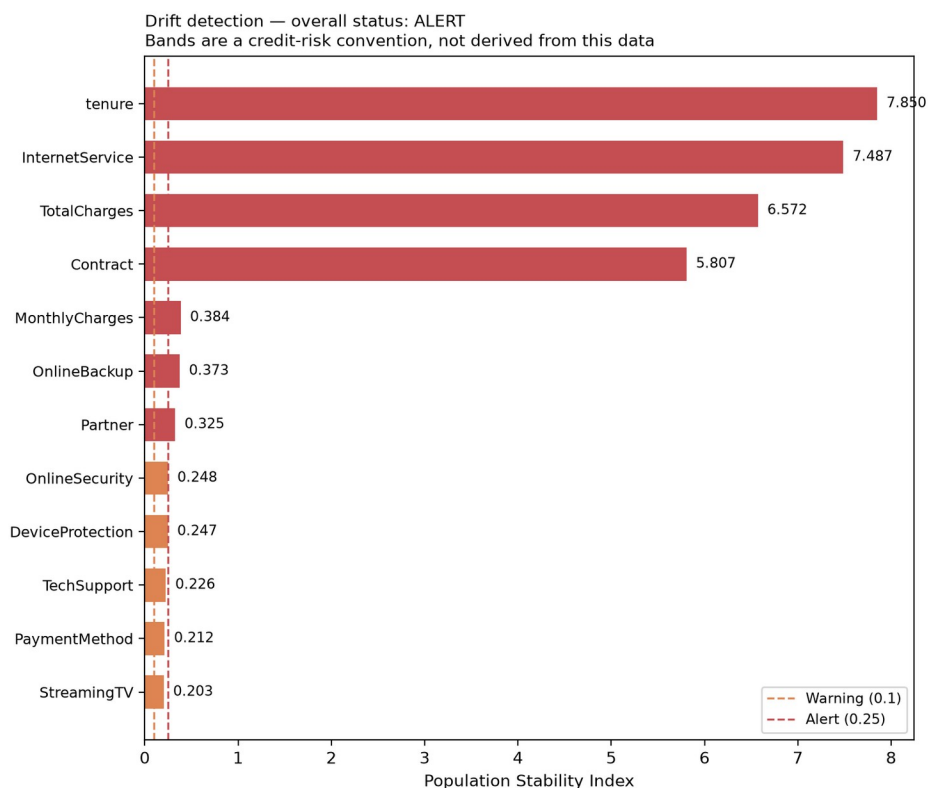


Figure 14. Drift detection on a simulated acquisition campaign. The detector reports ALERT on shifted data and STABLE on the unshifted holdout.

The detector is validated in **both directions**, which matters: a detector that always fires is as useless as one that never does. On the unshifted held-out set it flags 0 of 19 features and reports STABLE. On a deliberately shifted sample — mostly new month-to-month fibre customers, as an acquisition campaign would produce — it flags 15 of 19 with 7 at alert level and reports ALERT.

7.6 Feature engineering and hyperparameter search

Seven interpretable features were engineered and both models were grid-searched. Four arms were compared by cross-validation on the training split, so the contribution of each change is separable rather than confounded.

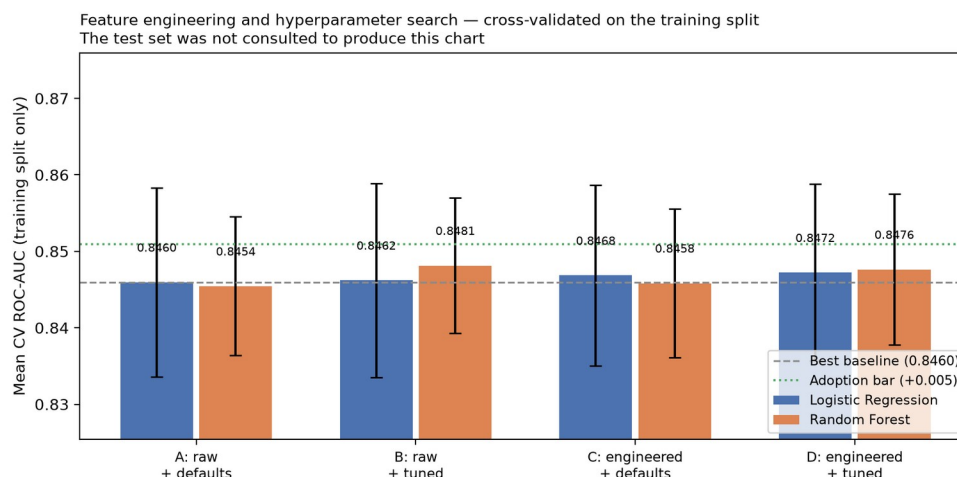


Figure 15. Four-arm comparison of feature engineering and hyperparameter search, cross-validated on the training split only.

Arm	Features	Hyperparameters	Best CV ROC-AUC
A	Raw 19	Hand-set (deployed)	0.8460
B	Raw 19	Grid-searched	0.8481
C	Raw + 7 engineered	Hand-set	0.8468
D	Raw + 7 engineered	Grid-searched	0.8476

Outcome: not adopted. The best arm gained +0.0022 mean CV ROC-AUC against an adoption bar of 0.005 that was fixed before the results were read — less than half the bar, and well inside the ± 0.0124 cross-validation standard deviation. The deployed model is unchanged. This is a result rather than a failure: it says the signal in this dataset is largely exhausted by the raw features and sensible defaults. Adopting a difference indistinguishable from noise would have added a permanent second transformation to keep consistent between training and serving, for nothing.

8. The Working Application

8.1 Design principles

- Artifacts are loaded once per process with `st.cache_resource`, using paths resolved relative to the application file. The application never trains anything at startup.
- Every control is built from `feature_schema.json`, so the deployed form cannot drift away from the trained pipeline.
- A token-based design system provides light and dark themes with an in-app toggle. The charts consume the same tokens, so they follow the theme rather than remaining light-mode islands.
- No external font, stylesheet or script is referenced. The page is entirely self-contained, which removes a supply-chain and privacy surface.
- Errors show a user-safe message containing no path, stack frame or internal identifier; the technical detail goes to the server log.

8.2 Application home screen

Churn Intelligence
Retention decision support

☐ Dark mode

Decision threshold
Flag a customer at or above 0.50
Documented default (0.50) — an assumption, not an optimum.

Demonstration cases
Load a documented profile
Case A — established two-year c
A long-tenure customer on a two-year contract paying by bank transfer, holding support and security add-ons. Used to demonstrate the form with a settled account profile.
Load this profile

Application version 1.1.0

Customer Churn Intelligence
Estimates whether a customer shows elevated churn risk, so retention specialists can prioritise accounts for a structured human review.

Model: **Logistic Regression** | Version: **1.1.0** | Dataset: **IBM Telco sample (fictional)** | Decision support: **Human review required**
Application version 1.1.0 — automated deployment verification

Data and use disclaimer. This application is trained on IBM's publicly published Telco Customer Churn sample, which represents a **fictional** telecommunications company. It is not actual observed commercial customer data. Outputs are decision support only; they may support prioritisation for human review and must not be used to change prices, modify or terminate contracts, deny service, or make any customer-treatment decision automatically.

DEPLOYED MODEL Logistic Regression Selected by cross-validation	TEST ROC-AUC 0.841 Held-out set, measured once	TEST RECALL (CHURN) 0.783 Share of actual churners identified	DECISION THRESHOLD 0.50 Documented default, not optimised
---	--	---	---

Single customer assessment | Batch scoring — retention work queue

CUSTOMER PROFILE | Demographic attributes as recorded

Gender: ☐ Male ☒ Female | Senior citizen: ☐ Yes ☒ No

Has partner: ☐ Yes ☒ No | Has dependents: ☐ Yes ☒ No

SERVICES | Add-ons follow the subscription they depend on

Phone service: ☐ No ☒ Multiple lines | Multiple lines: ☐ No phone service ☒ Yes

Internet service: ☐ No ☒ Yes

No assessment yet
Complete the customer details and select **Generate churn assessment**. The result shows a probability, a risk band and a suggested review action for a retention specialist to confirm.

Figure 16. The deployed application at <https://krish21may-churn.hf.space>, showing the business purpose, model and version badges, the fictional-data disclaimer, and measured held-out performance.

8.3 Input form and logical dependencies

Controls are provided for all nineteen predictors, grouped into customer profile, services, contract and billing, and charges and tenure. Logical dependencies are enforced so the form cannot produce a

combination the model never saw: with no phone service, MultipleLines locks to "No phone service"; with no internet service, every internet add-on locks to "No internet service".

8.4 A completed assessment

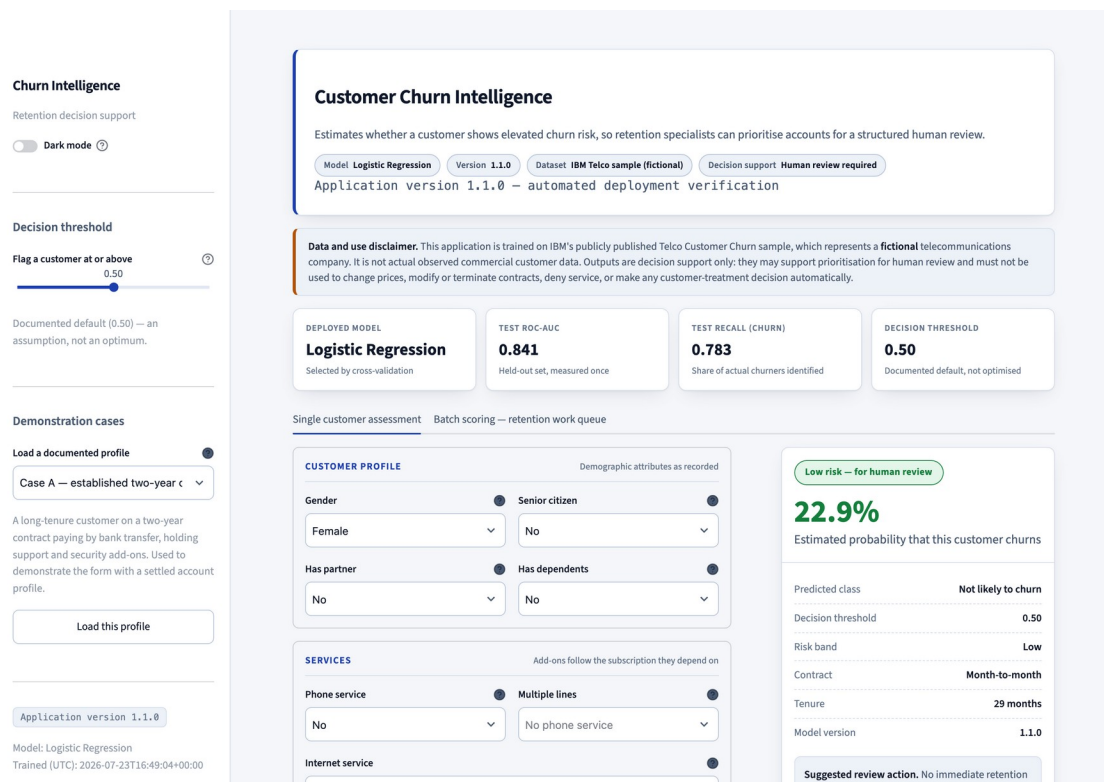


Figure 17. A completed assessment on the live Space. The result card shows the probability, risk band, predicted class, decision threshold, model version and a cautiously worded suggested review action.

The same input returns **22.9%** in the local Python environment, the local Docker container and the deployed Space. This cross-environment agreement is the practical justification for pinning scikit-learn to the exact training version.

8.5 Per-prediction contribution analysis



Figure 18. The contribution breakdown on the live Space, showing which attributes moved the score and in which direction, with the causal-language disclaimer in the chart title.

The chart is titled "What moved this model's score (association within the training sample, not causes)". The wording is deliberate: the interface must never imply that changing an attribute would change the customer's behaviour.

8.6 Batch scoring and the retention work queue

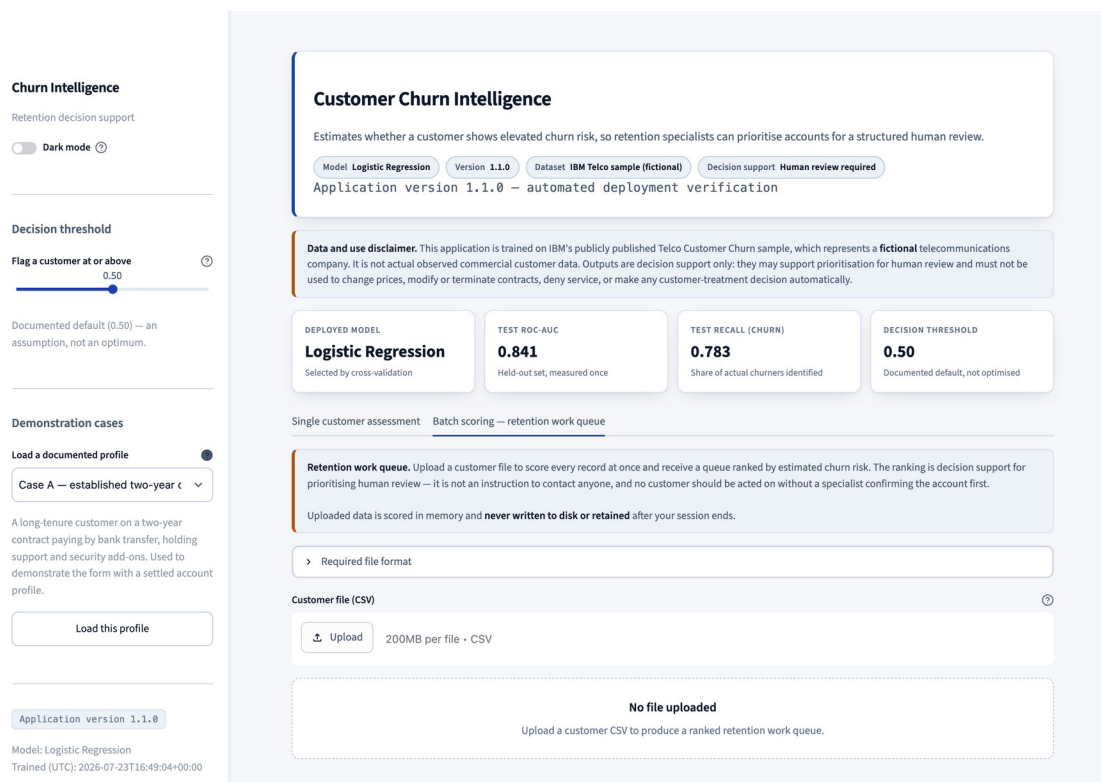


Figure 19. The batch scoring tab, which turns an uploaded customer file into a ranked retention work queue with a CSV export.

A customer CSV is validated against the feature schema before any scoring begins, so a malformed file produces per-column errors rather than a partial result that someone might trust. One thousand customers are scored and ranked in 0.01 seconds. Uploaded data is scored in memory and never written to disk.

8.7 Local testing performed

- All dependencies installed into a Python 3.11 virtual environment matching the container runtime.
- The application run locally and exercised with both documented demonstration profiles and with manual input combinations.
- Both themes verified in a browser, including the locked-dependency behaviour and the adjustable threshold.
- A Streamlit smoke test in the release gate starts the real server and requires HTTP 200 from its health endpoint.
- The Docker image built and run locally, confirmed to answer its health check and to reproduce the local prediction exactly.

8.8 A note on the evidence screenshots

Figures 16 to 19 were captured by scripts/capture_screenshots.py, which drives a headless browser to <https://krish21may-churn.hf.space>. The equivalent local captures held in docs/screenshots/ are byte-for-byte identical to them. This is expected rather than suspicious. The Space runs the same container image at the

same viewport and device scale factor, against the same pinned dependencies and the same deterministic pipeline, so the rendered page is pixel-identical. The identity is itself evidence that the deployed application reproduces the local one exactly. Because a page screenshot excludes the browser address bar, the capture script — which is committed to the repository — is the record of which URL each image was taken from.

9. GitHub Repository Structure

9.1 Repository

<https://github.com/krish2105/SDAIM-Customer-Churn>

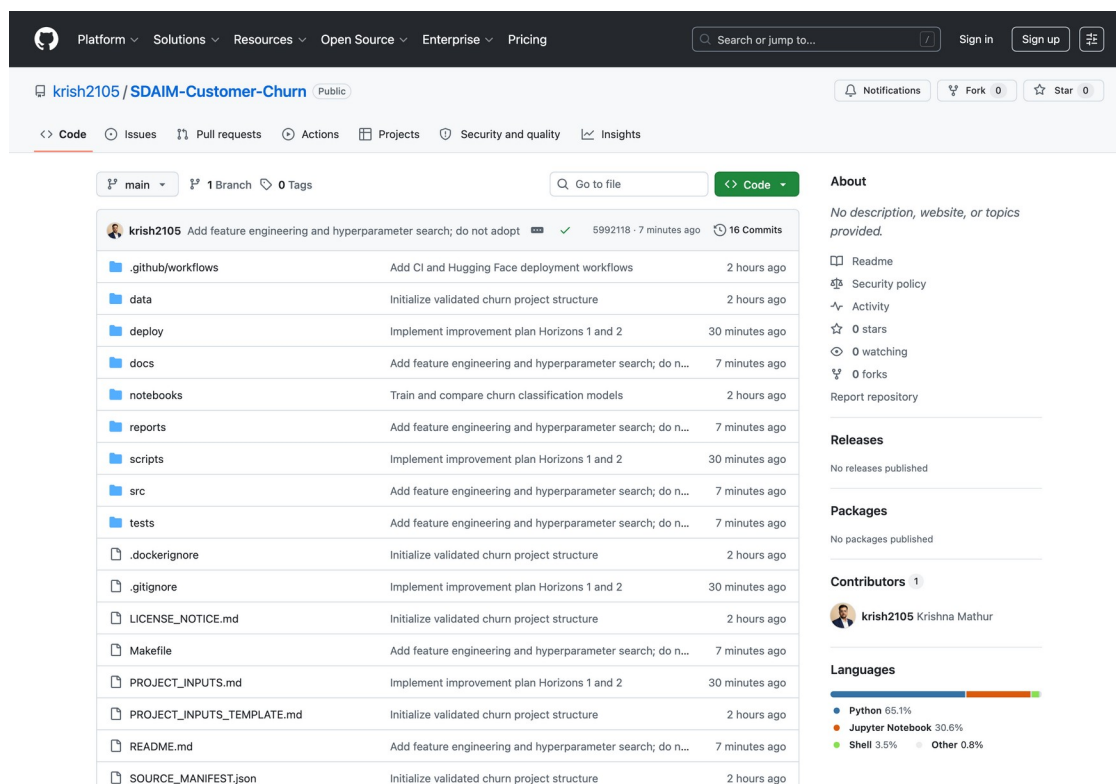


Figure 20. The public GitHub repository containing the complete application, deployment files, tests and documentation.

9.2 Structure

The brief suggests a minimal structure of `app.py`, `model.joblib`, `requirements.txt`, `README.md` and `.github/workflows/deploy.yml`. All five are present. They sit inside a fuller structure that separates training code from the deployment package, because only the deployment package should ever reach the Space.

SDAIM-Customer-Churn/	
├── README.md	Executive summary, results, links
├── LICENSE_NOTICE.md	Dataset and dependency licensing
├── Makefile	Every command, runnable from the root
├── requirements-dev.txt	Development and analysis dependencies
├── data/	
│ ├── raw/	Immutable, SHA-verified dataset
│ └── processed/	Reproducible split exports
├── src/	config, schemas, validation, EDA, train, evaluate, fairness, calibration, threshold, drift, tracking, features, tuning
├── notebooks/	01_eda_and_modeling.ipynb (executed)
├── reports/	figures, tables and analysis reports
├── deploy/	<-- everything that ships to the Space
│ ├── app.py	Streamlit application
│ └── theme.py	Design tokens and scoped CSS

— charts.py	Themed matplotlib charts
— explain.py	Per-prediction contributions
— batch.py	Batch scoring and work queue
— rationale.py	Guardrailed retention brief
— Dockerfile	python:3.11-slim, non-root, port 7860
— requirements.txt	Pinned runtime dependencies
— README.md	Space configuration (YAML front matter)
— artifacts/	
— model_pipeline.joblib	
— model_metadata.json	
— feature_schema.json	
— reference_rates.json	
— model_card.md	
— tests/	115 automated tests
— scripts/	bootstrap, validate, train, test, docker, secret scan, release verification
— docs/	Audit trail and supporting documentation
— .github/	
— workflows/	
— ci.yml	Validation on every push and pull request
— deploy.yml	Automated deployment to the Space

9.3 The deployment package

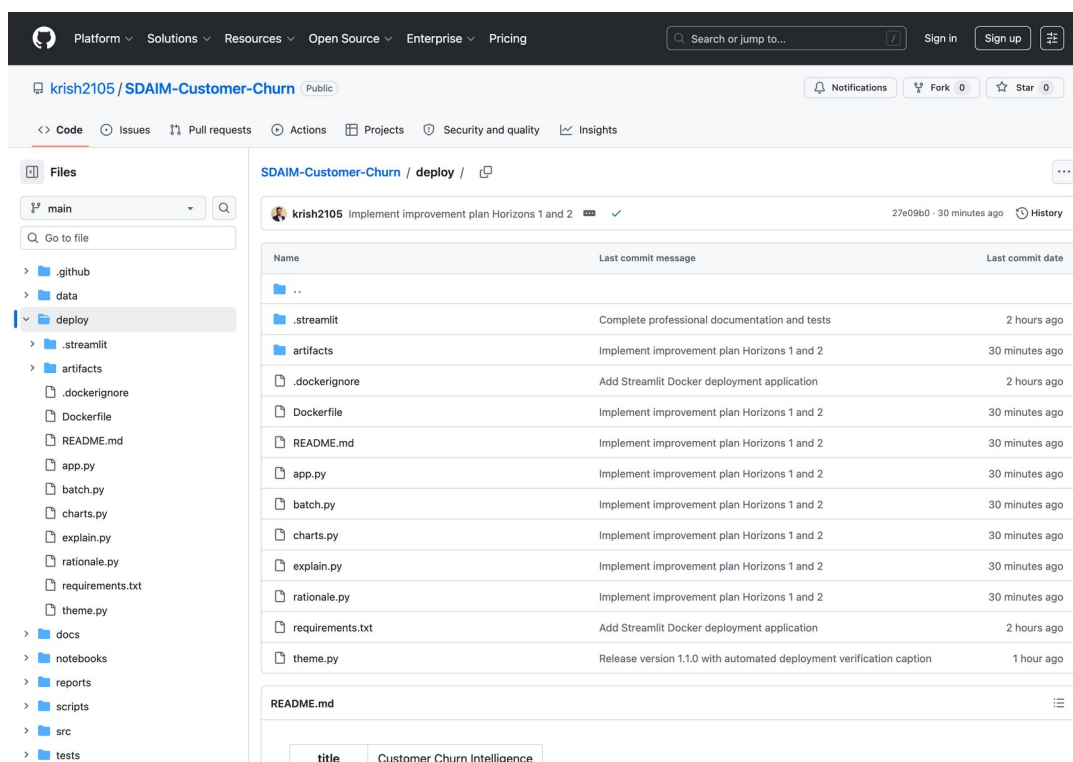


Figure 21. The `deploy/` directory — the only part of the repository synchronised to the Hugging Face Space.

Keeping the deployment package in its own directory means the training code, the raw dataset, the notebooks, the tests and the reports never enter the container. The image stays small and its attack surface minimal. A test asserts that the application never imports the training package.

9.4 Version control practice

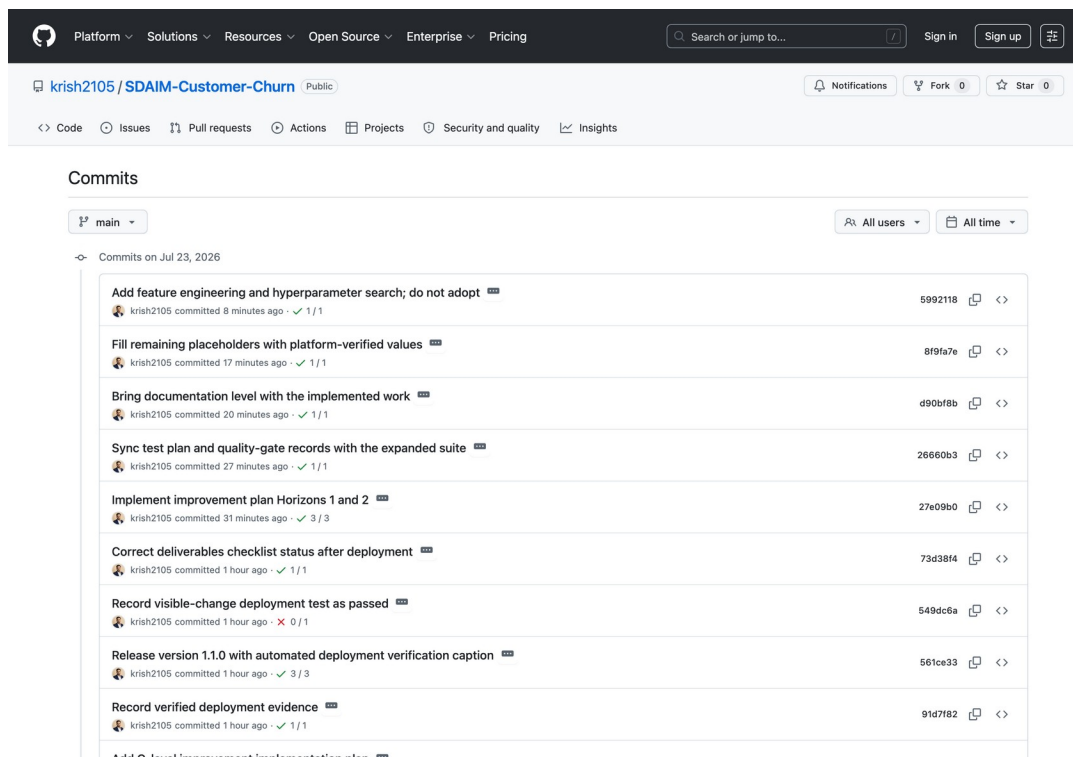


Figure 22. Commit history showing incremental development. Each commit was made only after the full test suite and the secret scan passed.

Before every commit the full test suite and the secret scanner are run, and the staged file list is reviewed. The release-verification script additionally fails if any tracked file matches a credential-bearing filename.

10. The GitHub Actions Workflows

Two workflows are configured. One validates every change; the other deploys.

10.1 Continuous integration — ci.yml

Triggered on pull requests and on pushes to main. It performs eight steps:

14. Check out the repository, with Git LFS enabled.
15. Set up Python 3.11, matching the container runtime exactly.
16. Install the development dependencies.
17. Validate the raw dataset with `--strict-sha`, so a missing Git binary fails the build rather than skipping the check.
18. Compile every source and test file.
19. Run the full test suite.
20. Run the secret scanner.
21. Verify that every required deployment artifact exists and that the Space README declares the Docker SDK on port 7860.

10.2 Deployment — deploy.yml

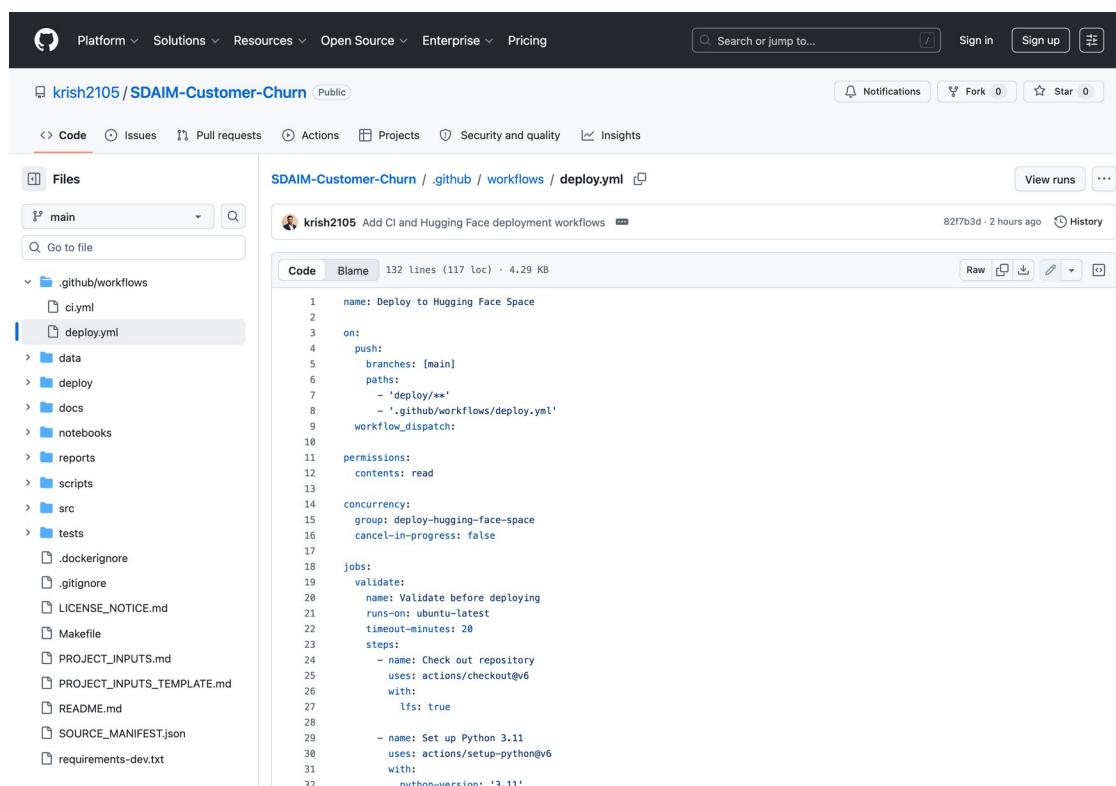


Figure 23. The deployment workflow as published in the repository.

The workflow is triggered by pushes to main that touch `deploy/**`, and can also be dispatched manually. Its design reflects several deliberate decisions:

Property	Value and rationale
Validation gate	A validate job runs the dataset check, the tests and the secret scan. The deploy job declares needs: validate, so a failing test means nothing ships.
Permissions	contents: read only — the least privilege the job can operate with.
Concurrency	A named concurrency group prevents two deployments from overlapping.
Configuration check	HF_SPACE_ID and HF_TOKEN are verified to exist, and the Space ID format is validated, before anything is pushed.
Credential handling	The token is read only as <code>\${{ secrets.HF_TOKEN }}</code> . It is never echoed, never interpolated into a URL, and never written to the job summary.
Sync action	huggingface/hub-sync@v0.2.1 with <code>repo_type: space</code> , <code>space_sdk: docker</code> and <code>subdirectory: deploy</code> .

A required input the specification omitted. The action's published definition was read rather than assumed, which revealed that `github_repo_id` is a required input. It is supplied as `${{ github.repository }}`. Without it the deployment job would have failed at run time.

10.3 Successful workflow execution

The screenshot shows the GitHub Actions interface for the repository `krish2105/SDAIM-Customer-Churn`. The workflow `Complete professional documentation and tests #1` is displayed, triggered by a push to the `main` branch. The execution status is **Success** with a total duration of **48s**. The workflow file `ci.yml` is shown, containing a single job `Validate, test and verify deployment package` that completed successfully in 43s. The left sidebar shows the workflow summary, and the top navigation bar includes links to Code, Issues, Pull requests, Actions, Projects, Security and quality, and Insights.

Figure 24. Successful continuous-integration run 30019016553. All ten steps completed green.

krish2105 / SDAIM-Customer-Churn

Deploy to Hugging Face Space

Release version 1.1.0 with automated deployment verification caption #3

Summary

All jobs

- Validate before deploying
- Sync deploy/ to the Space

Run details

- Usage
- Workflow file

Triggered via push 1 hour ago

krish2105 pushed → 561ce33 main

Status: Success

Total duration: 1m 11s

Artifacts: -

deploy.yml

on: push

Validate before deploying 47s

Sync deploy/ to the Space 16s

Figure 25. Successful deployment run. The validate job and the sync job both completed green.

krish2105 / SDAIM-Customer-Churn

Actions

All workflows

CI

Deploy to Hugging Face Space

Management

Caches

All workflows

Showing runs from all workflows

Filter workflow runs

15 workflow runs

Workflow	Event	Status	Branch	Actor
Add feature engineering and hyperparameter search; do not ...	CI #11: Commit 5992118 pushed by krish2105	Success	main	
Fill remaining placeholders with platform-verified values	CI #10: Commit 8f9fa7e pushed by krish2105	Success	main	
Bring documentation level with the implemented work	CI #9: Commit d90bf8b pushed by krish2105	Success	main	
Sync test plan and quality-gate records with the expanded s...	CI #8: Commit 26660b3 pushed by krish2105	Success	main	
Implement improvement plan Horizons 1 and 2	Deploy to Hugging Face Space #4: Commit 27e09b0 pushed by krish2105	Success	main	
Implement improvement plan Horizons 1 and 2	CI #7: Commit 27e09b0 pushed by krish2105	Success	main	
Correct deliverables checklist status after deployment	CI #6: Commit 73d38f4 pushed by krish2105	Success	main	
Record visible-change deployment test as passed	CI #5: Commit 549dc6a pushed by krish2105	Success	main	
Release version 1.1.0 with automated deployment verification...		Success		

Figure 26. The Actions history for the repository, showing repeated successful runs of both workflows.

10.4 The guard demonstrated failing before it was demonstrated passing

Stronger evidence than a first-time success. Before HF_SPACE_ID and HF_TOKEN were configured, deployment run 30019016510 behaved exactly as designed: the validate job passed every step, then the deploy job failed at the configuration check with the message "Repository variable HF_SPACE_ID is not configured", and skipped the sync, package-verification and summary steps rather than attempting a push with an empty repository id. The guard was shown to fire, then shown to pass once configured — the same standard applied to the secret scanner, which was verified against synthetic credentials.

11. Deployment on Hugging Face

11.1 The Space

<https://huggingface.co/spaces/krish21may/churn>

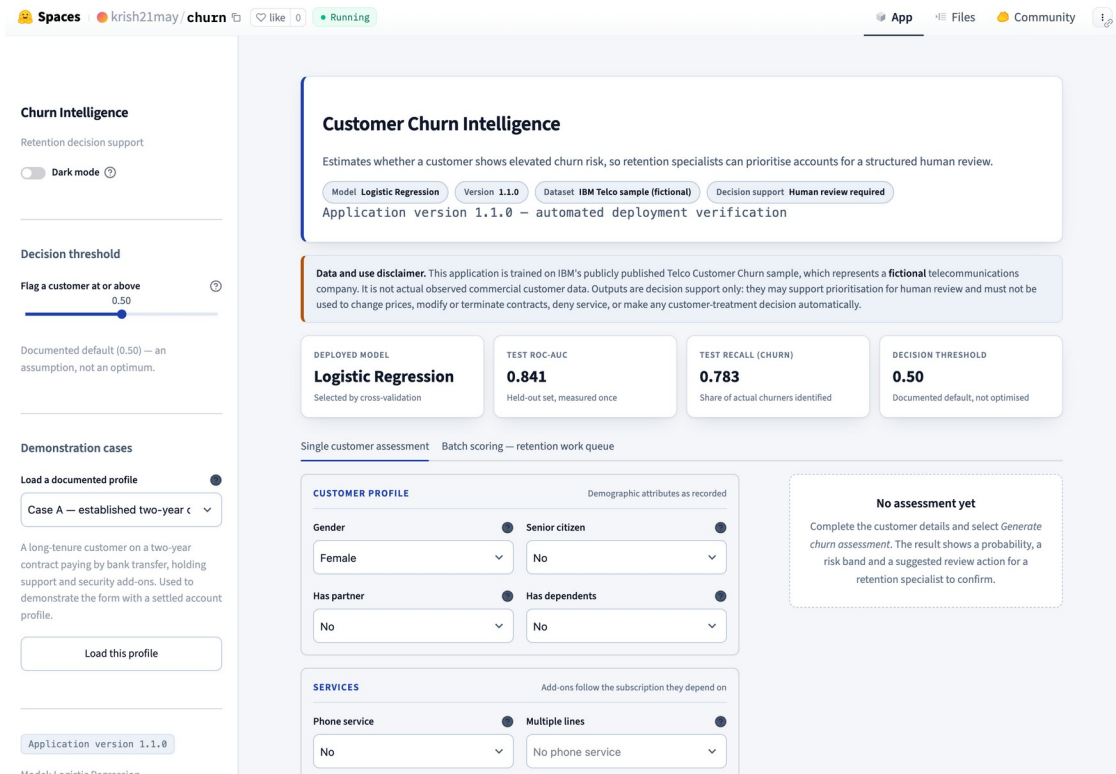


Figure 27. The deployed application running publicly on Hugging Face Spaces.

11.2 Space configuration

Property	Value
Space ID	krish21may/churn
SDK	Docker
Application port	7860
Visibility	Public
Base image	python:3.11-slim
Runtime user	Non-root (appuser, uid 1000)
Runtime status	RUNNING

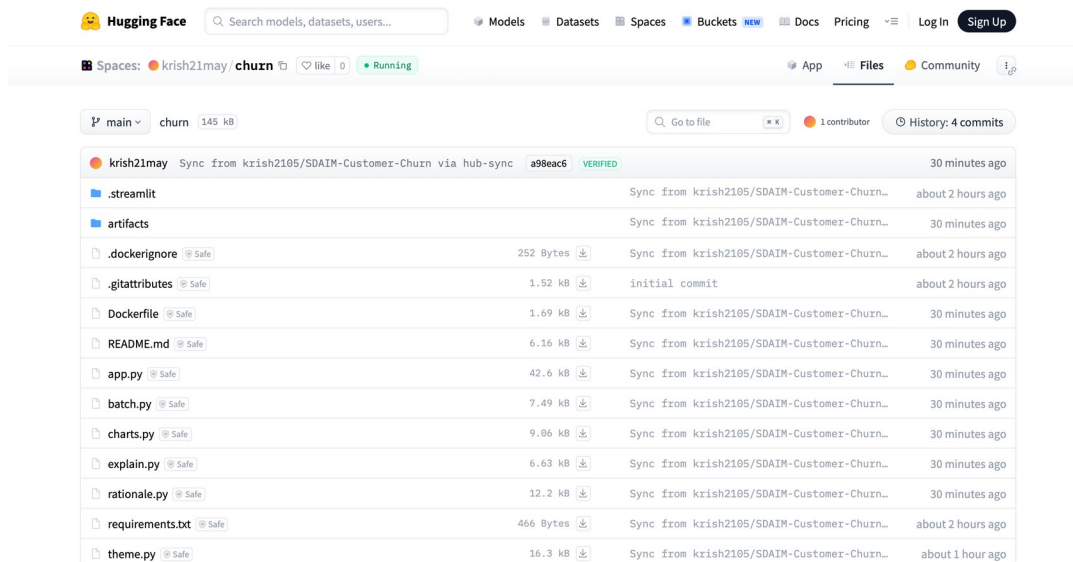


Figure 28. The files present in the Space, synchronised automatically from the `deploy/` directory of the GitHub repository.

11.3 The container

The Dockerfile creates a non-root application user, installs the pinned runtime dependencies as a separately cached layer, copies only the runtime files and the artifacts, grants the non-root user read access, exposes port 7860 and declares a health check. No credential appears anywhere in the image, and a test asserts that.

```
FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1 PYTHONNOINSTDIRECTORY=1 PIP_NO_CACHE_DIR=1
RUN useradd --create-home --uid 1000 appuser
WORKDIR /app
COPY requirements.txt /app/requirements.txt
RUN python -m pip install -r /app/requirements.txt
COPY app.py theme.py charts.py explain.py batch.py rationale.py /app/
COPY artifacts /app/artifacts
USER appuser
EXPOSE 7860
CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0",
    "--server.port=7860", "--server.headless=true"]
```

11.4 Verified deployment evidence

Gate	Status	Evidence
Repository pushed	PASS	Public, main branch, 19 commits at the time of writing
CI run	PASS	Run 30019016553, all ten steps green
Deployment run	PASS	Run 30020847651, both jobs green
Space build	PASS	Reached runtime stage RUNNING

Gate	Status	Evidence
Live prediction	PASS	22.9%, Low risk band
Visible-change redeployment	PASS	Run 30023117028, version 1.1.0 confirmed live

12. How the Automated Deployment Works

12.1 The chain of events

A developer pushes a change to main that touches the deploy/ directory. From that moment nothing is manual:

22. GitHub detects the push and evaluates the workflow's path filter. A change confined to documentation or training code does not trigger a deployment.
23. The validate job checks out the repository, installs dependencies, verifies the dataset against its Git blob SHA, runs the full test suite and scans for credentials.
24. Only if that job succeeds does the deploy job begin, because it declares needs: validate.
25. The deploy job verifies that the HF_SPACE_ID repository variable and the HF_TOKEN secret both exist, and that the Space ID has the expected owner/name form. A missing value fails here with an actionable message rather than producing an obscure push error.
26. It verifies that every required file is present in the deployment package.
27. The huggingface/hub-sync action authenticates with the token and synchronises the contents of deploy/ into the Space repository.
28. Hugging Face detects the new commit in the Space and rebuilds the Docker image from the synchronised Dockerfile.
29. When the build succeeds, the Space restarts and serves the new version on port 7860.

12.2 Authentication

Authentication uses a fine-grained Hugging Face access token scoped to write to the single target Space and nothing else. It is stored as an encrypted GitHub Actions secret named HF_TOKEN and is never present in the repository, the container image or any log. The Space identifier is stored separately as a repository variable named HF_SPACE_ID, because it is a public identifier rather than a secret and printing it aids diagnosis.

A green workflow is not a successful deployment. The sync step reports that files were transferred. The Space build happens afterwards and can fail on its own. This project therefore treats them as separate claims requiring separate evidence, and the workflow summary says so explicitly rather than implying the deployment succeeded.

12.3 The visible-change test

To prove the automation genuinely works rather than having succeeded once by coincidence, a deliberate visible change was made and traced end to end:

30. The application version was raised from 1.0.0 to 1.1.0 in src/config.py.
31. The model was retrained. The held-out metrics came back identical — ROC-AUC 0.8414, recall 0.7834 — confirming determinism at random_state=42. Only the version and training timestamp changed.

32. A visible caption was added to the application header, rendered from the artifact metadata rather than hard-coded so it can never disagree with the deployed version.
33. The tests and the secret scan were run locally, then the change was committed and pushed.
34. A second deployment run started automatically from the path filter, with no manual dispatch.
35. The Space rebuilt, and the live application now displays "Application version 1.1.0 — automated deployment verification" alongside the version badge.
36. Predictions were confirmed still working after the redeployment.

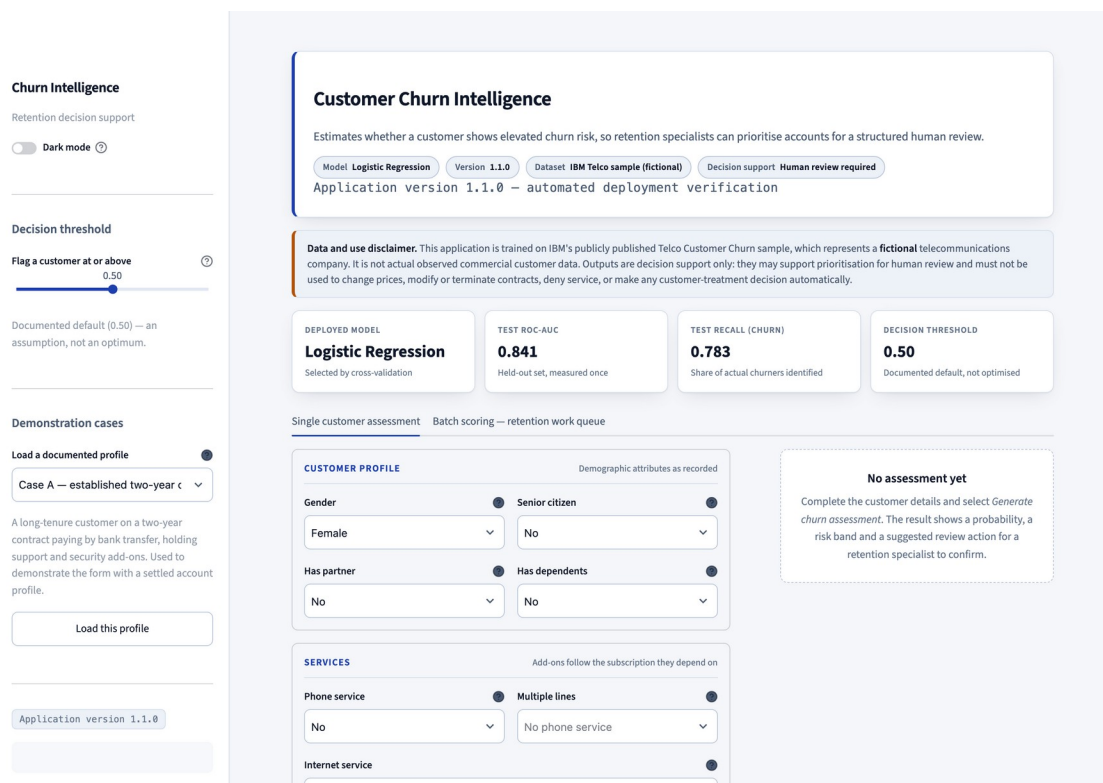


Figure 29. The live application showing version 1.1.0 and the automated deployment verification caption, reached with no manual deployment step.

This satisfies Part 4.4 of the brief: a change pushed to main reached the live public application automatically.

13. Security and Governance

13.1 Credential handling

Item	Type	Location
HF_TOKEN	Encrypted Actions secret	Repository settings — value never retrievable after creation
HF_SPACE_ID	Repository variable	Repository settings — a public identifier, deliberately not a secret

The token is fine-grained and scoped to write to the single target Space. If it leaked, it could affect nothing else in the account.

13.2 Secret scanning, and its own verification

A scanner runs locally, in continuous integration, and again in the deployment workflow before anything is pushed. It detects Hugging Face tokens, GitHub personal access tokens, AWS access keys, secret keys and session tokens, private-key headers and bearer tokens, and fails on the mere presence of credential-bearing filenames.

The scanner reports only the file path and the pattern category, never the matched text — a scanner that echoed its findings would leak the secret into continuous-integration logs, which are often more widely readable than the repository itself. **The scanner was itself verified** by placing a file containing four synthetic credentials into the project: both the shell scanner and the pytest equivalent correctly failed, then passed again once the file was removed.

13.3 Container hardening

- Runs as a non-root user created explicitly in the image.
- Built from a slim base image containing only the runtime files and artifacts.
- No credential is present in any layer, asserted by test.
- XSRF protection enabled; usage statistics disabled.
- No external resource is fetched at build or request time.

13.4 Application-level controls

- No stack trace, filesystem path or internal identifier is ever shown to a user.
- Every value rendered into the interface passes through HTML escaping.
- No customer data is persisted; batch uploads are scored in memory and discarded.

13.5 Governance position

What this model must never do. It must not autonomously change prices, terminate or modify contracts, deny service, target customers unfairly, or make any financial or customer-treatment decision. Every output requires

human review before a customer is contacted. A churn score is a statistical association, not a judgement about a person.

13.6 Known security gaps

Recorded rather than omitted: there is no automated dependency-vulnerability scanning, no container image scanning, and no signed commits or images. Adding pip-audit and Trivy to the CI job is the recommended next step.

14. Testing and Quality Assurance

14.1 Test suite

Module	Tests	What it guards
test_data_validation.py	13	The dataset contract — schema, counts, domains, blob SHA
test_model_artifact.py	12	Artifact integrity, metadata realism, fresh-process loading
test_prediction.py	11	Prediction behaviour, column-order enforcement, determinism
test_deployment_files.py	16	Deployment configuration, workflow scope, secret hygiene
test_analysis.py	33	Fairness, calibration, threshold, drift and feature engineering
test_app_features.py	30	Explainability, batch scoring, retention-brief guardrails
Total	115	All passing

14.2 Tests that guard against silent corruption

- **Column-order enforcement.** The input columns are reversed and an identical probability is required. Positional rather than name-based selection is a classic silent bug: the pipeline still runs and still returns a plausible number, just a wrong one.
- **Contribution reconstruction.** The explanation must reproduce the model's own score to floating-point precision across 25 real customers. If it ever drifts, the chart is lying and must not be displayed.
- **Runtime dependency coverage.** Each runtime module's abstract syntax tree is parsed and every third-party import checked against the pinned requirements. A hand-maintained list would go stale; this cannot.
- **Batch and single-record agreement.** The two scoring paths must return identical probabilities, or the work queue could disagree with the form.
- **Drift detector, both directions.** It must fire on a shifted sample and stay silent on the unshifted holdout.

14.3 Quality gates

A release-verification script runs ten non-interactive gates and records the actual result of each: dataset validation, syntax compilation, artifact reload in a fresh process, the test suite, a Streamlit smoke test against the real server, the secret scan, required-file verification, the Docker build, a container run with a health check, and a Git staged-file review.

Latest run: 10 PASS, 0 FAIL, 0 NOT RUN.

14.4 Defects found and fixed by these gates

Recorded because each was a genuine defect that the process caught rather than a rough edge that was tidied:

37. The drift detector produced a false positive on the unshifted control set, because an escalation rule let a chi-squared p-value override the Population Stability Index. At $n \approx 1,400$ the test flags differences far too small to matter. PSI alone now governs the status, matching what the module's documentation had always said.
38. The prohibited-language filter in the retention brief matched the word "causes" inside its own disclaimer. The pattern now matches assertions of cause rather than the noun, and a test asserts that the fallback text passes the filter it enforces.
39. A placeholder check matched the word "actual" inside the legitimate phrase "not actual observed commercial customer data".
40. The Streamlit configuration set `enableCORS = false` alongside `enableXsrfProtection = true`, a combination Streamlit rejects.
41. Two feature-engineering tests encoded a wrong expectation — that no engineered feature may be null. `AvgMonthlySpend` is null for exactly the eleven documented blank-`TotalCharges` rows, deliberately, so the pipeline's median imputer handles it. The tests were corrected, not the code.

15. Limitations and Future Work

15.1 Limitations

42. **Fictional data.** Performance on any live population is unknown and must be revalidated before operational use.
43. **No time dimension.** A single cross-section means drift and seasonality cannot be assessed and no monitoring baseline exists.
44. **Class imbalance limits precision.** At 0.7834 recall, roughly half the flagged accounts are false positives.
45. **Probabilities are not calibrated.** The model overstates churn probability by about 0.15. Ranking is reliable; magnitude is not.
46. **The threshold is an assumption.** 0.50 is cost-optimal at roughly 3:1, a ratio the business never supplied.
47. **Protected attributes are retained.** Removal is recommended and was measured to be nearly free, but was deferred to preserve the published evidence trail.
48. **Association, never causation.** The model cannot say what would happen if a customer's contract or services were changed.
49. **No measured business impact.** No revenue or churn-reduction effect has been demonstrated.

15.2 Future work

Initiative	Why it matters
Remove the protected attributes	Measured to cost +0.0008 ROC-AUC; removes an unforced governance risk
Deploy the calibrated model	Isotonic calibration cuts ECE from 0.1503 to 0.0194 at no cost to ranking
Obtain the business cost ratio	One question resolves the threshold entirely
FastAPI prediction service	Decouples the model from the interface so other systems can consume it
Database and feature store	Persists scores and outcomes, enabling a genuine feedback loop
Scheduled retraining with gates	Retrain on a trigger; block promotion on a metric regression
Live monitoring and alerting	Routes drift and performance decay to a real channel
A/B evaluation of interventions	The only route to a substantiated business-value claim
Dependency and image scanning	Closes the known security gap

The honest ceiling of the current evidence. Every claim about business value remains unproven until the A/B evaluation exists. Until a control group has been observed, this platform demonstrates technical feasibility and may support prioritisation for human review. Stating that plainly is a strength of the work, not a hedge.

16. Conclusion

This project delivers a complete, reproducible path from a provenance-verified dataset to an automatically deployed public application. The selected model reaches 0.8414 ROC-AUC and 0.7834 recall on held-out data, identifying 293 of 374 churners. The application is live, the deployment is automated, and a deliberate visible change was traced from a local commit to the running Space with no manual step.

What we would emphasise as much as the metrics is the discipline around them. The selection rule was written before the test set was examined. The fairness audit was accompanied by a counterfactual that changed its own recommendation. The calibration outcome was predicted before it was measured. The drift detector was validated in both directions. Feature engineering and hyperparameter search were carried out, measured, and then rejected on the evidence because the gain did not clear a bar set in advance.

Equally important is what this platform does not do. It makes no financial claim, automates no customer-treatment decision, and states its limitations in the application itself as well as in this report — including the ones that remain open. It demonstrates technical feasibility and may support prioritisation for human review. That is the honest extent of what the evidence supports.

17. References

50. IBM. Telco Customer Churn sample. <https://github.com/IBM/telco-customer-churn-on-icp4d> (Apache-2.0).
51. IBM Documentation. Samples: Telco customer churn. <https://www.ibm.com/docs/en/cognos-analytics>
52. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
53. Mitchell, M. et al. (2019). Model Cards for Model Reporting. *Proceedings of FAT* '19*, 220–229.
54. Streamlit documentation. <https://docs.streamlit.io>
55. Hugging Face. Spaces — Docker SDK. <https://huggingface.co/docs/hub/spaces-sdks-docker>
56. GitHub. GitHub Actions documentation. <https://docs.github.com/actions>
57. Hugging Face. hub-sync action. <https://github.com/huggingface/hub-sync>
58. W3C. Web Content Accessibility Guidelines (WCAG) 2.1. <https://www.w3.org/TR/WCAG21/>

18. Team Contributions

Member	Student ID	Primary contribution
Krishna Mathur	AS25DXB018	Platform, Docker packaging, CI/CD workflows, security controls and deployment

Member	Student ID	Primary contribution
Yash Petkar	AS25DXB021	Modelling rigour — fairness audit, calibration analysis, experiment tracking and registry
Atharva Soundankar	AS25DXB020	Application development, explainability, threshold analysis, batch scoring and drift detection

All three members contributed to the exploratory analysis, the report, the evidence pack and the demonstration.

Customer Churn Intelligence and Retention Decision-Support Platform · Version 1.1.0
github.com/krish2105/SDAIM-Customer-Churn · huggingface.co/spaces/krish21may/churn